Lec 09 Policy Function Approximation (or Policy Gradient)

Basic idea of policy gradient
- Differences between tabular and function representations
- Basic idea

Metrics to define optimal policies
- The average value
  - How to select the distribution d? There are two cases.
- The average reward
  - An equivalent definition

Gradients of the metrics

Gradient-ascent algorithm (REINFORCE)
- Rseudocode

Basic idea of policy gradient

Previously, policies have been represented by tables, where each entry of the table is indexed by a state and a action. We can directly access or change a value in the table.

	a_1	a_2	a_3	a_4	a_5
s_1	$\pi(a_1\|s_1)$	$\pi(a_2\|s_1)$	$\pi(a_3\|s_1)$	$\pi(a_4\|s_1)$	$\pi(a_5\|s_1)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$\pi(a_1\|s_9)$	$\pi(a_2\|s_9)$	$\pi(a_3\|s_9)$	$\pi(a_4\|s_9)$	$\pi(a_5\|s_9)$

Now, policies can be represented by parameterized functions:

$$\pi(a|s,\theta)$$

where $\theta \in \mathbb{R}^m$ is a parameter vector. The function can be, for example, a neural network.

Advantage: when the state space is large, the tabular representation will be of low efficiency in terms of storage and generalization.

Differences between tabular and function representations

> - **First, how to define optimal policies?**
> - When represented as a table, a policy π is optimal if it can maximize every state value.
> - When represented by a function, a policy π is opyimal if it can maximize **certain scalar metrics**.
>
> - **Second, how to access the probability of an action?**
> - In the tabular case, the probability of taking a at s can be directly accessed by looking up the tabular policy.
> - In the case of function representation, we need to calculate the value of $\pi(a|s,\theta)$ given the function structure and parameter.
> - **Third, how to update policies?**
> - When represented by a table, a policy π can be updated by directly changing the entries in the table.
> - When represented by a parameterized function, a policy π cannot be updated in this way anymore. Instead, it can only be updated by changing the parameter θ.

Basic idea

First, metrics (or objective functions) to define optimal policies: $J(\theta)$

Second, gradient-based optimization algorithms to search for optimal policies:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t)$$

Metrics to define optimal policies

There are two metrics. The first metric is the average state value or simply called **average value**, and the second metric is average one-step reward or simply **average reward**.

The average value

The metric is defined as

$$\bar{v_\pi} = \sum_{s \in \mathcal{S}} d(s) v_\pi(s)$$

where

- $\bar{v}_\pi$ is a **weighted average of the state values**.
- $d(s) \geq 0$ is the **weight** for state s.
- Since $\sum_{s \in \mathcal{S}} d(s) = 1$, we can interpret $d(s)$ as a **probability distribution**. Then, the metric can be written as

$$\bar{v}_\pi = \mathbb{E}[v_\pi(S)]$$

where $S \sim d$.

How to select the distribution d? There are two cases.

The first case is that d is independent of the policy π.

- In this case, we specifically denote d as d_0 and $\bar{v}_\pi$ as $\bar{v}_\pi^0$.
- How to select d_0?
  - One trivial way is to treat all the states equally important and hence select $d_0(s) = 1/|\mathcal{S}|$.
  - Another important case is that we are only interested in a specific state s_0. For example, the episodes in some tasks always start from the same state s_0. Then, we only care about the long-term return starting from s_0. In this case,

$$d_0(s_0) = 1, d_0(s \neq s_0) = 0$$

The second case is that d depends on the policy π.

- A common way to select d as $d_\pi(s)$, which is the **stationary distribution** under π.
- The interpretation of selecting d_π is as follows:

- If one state is frequently visited in the long run, it is more important and deserves more weight.
- If a state is hardly visited, then we give it less weight.

The average reward

In particular, the metric is

$$\bar{r_\pi} = \sum_{s \in \mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}[r_\pi(s)]$$

where $S \sim d_\pi$. Here,

$$r_\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) r(s,a)$$

is the average of the one-step immediate reward that can be obtained starting from state s, and

$$r(s,a) = \mathbb{E}[R|s,a] = \sum_r r p(r|s,a)$$

- The weight d_π is the stationary distribution.
- As its name suggests, $\bar{r_\pi}$ is simply a weighted average of the one-step immediate rewards.

An equivalent definition

The average simple-step reward along the trajectory is

$$\begin{aligned} &\lim_{n \to \infty} \frac{1}{n} \mathbb{E}[R_{t+1} + R_{t+2} + \cdots + R_{t+n} | S_t = s_0] \\ &= \lim_{n \to \infty} \frac{1}{n} \mathbb{E}[\sum_{k=1}^{n} R_{t+k} | S_t = s_0] \end{aligned}$$

where s_0 is the starting state of the trajectory.

An important property is that the two definition of $\bar{r_\pi}$ are equivalent:

$$
\begin{aligned}
\lim_{n\to\infty}\frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n}R_{t+k}|S_t=s_0\right] &= \lim_{n\to\infty}\frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n}R_{t+k}\right] \\
&= \sum_{s} d_\pi(s) r_\pi(s) \\
&= \bar{r}_\pi
\end{aligned}
$$

> **Remarks:**
>
> - All these metrics are function of π. Since π is parameterized by θ, these metrics are functions of θ. Therefore we can search for the optimal values of θ to maximize these metrics.
> - Intuitively, $\bar{r_\pi}$ is more short-sighted because it merely considers the immediate rewards, whereas $\bar{v_\pi}$ consider the total reward overall steps.
> - However, the metrics are equivalent to each other. In the discounted case where $\gamma < 1$, it holds that
>
> $$\bar{r_\pi} = (1-\gamma)\bar{v_\pi}$$

Gradients of the metrics

Summary of the results about the gradients:

$$\nabla_\theta J(\theta) = \sum_{s\in\mathcal{S}}\eta(s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a|s,\theta)q_\pi(s,a)$$

where

- $J(\theta)$ can be $\bar{v}_\pi$, $\bar{r}_\pi$, or $\bar{v}_\pi^0$
- "$=$" may denote **"strict equality", "approximation", or "proportional to"**
- η is a distribution or weight of the states

Some specific results:

$$
\begin{aligned}
\nabla_\theta \bar{r}_\pi &\simeq \sum_s d_\pi(s) \sum_a \nabla_\theta \pi(a|s,\theta) q_\pi(s,a) \\
\nabla_\theta \bar{v}_\pi &= \frac{1}{1-\gamma} \nabla \theta \bar{r}_\pi \\
\nabla \theta \bar{v}_\pi^0 &= \sum_{s \in \mathcal{S}} \rho_\pi(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a)
\end{aligned}
$$

A compact and useful form of the gradient:

$$
\begin{aligned}
\nabla_\theta J(\theta) &= \sum_{s \in \mathcal{S}} \eta(s) \sum_{a \in \mathcal{A}} \nabla_\theta \pi(a|s,\theta) q_\pi(s,a) \\
&= \mathbb{E}[\nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)]
\end{aligned}
$$

where$S \sim \eta$ and $A \sim \pi(A|S,\theta)$.

Because we need to calculate$\ln \pi(a|s,\theta)$, we must ensure that for all s, a, θ

$$
\pi(a|s,\theta) > 0
$$

This can be archived by using **softmax functions** that can normalize the entries in a vector **from** $(-\infty, +\infty)$ **to** $(0, 1)$.

Then, the policy function has the form of

$$
\pi(a|s,\theta) = \frac{e^{h(s,a,\theta)}}{\sum_{a' \in \mathcal{A}} e^{h(s,a',\theta)}},
$$

where $h(s,a,\theta)$ is another function.

> **Remarks:**
>
> - Such a form based on the softmax function can be realized by a neural network whose input is s and parameter is θ. The network has $|\mathcal{A}|$ outputs, each of which corresponds to $\pi(a|s,\theta)$ for an action a. The activation function of the output layer should be softmax.
> - Since $\pi(a|s,\theta) > 0$ for all a, the parameterized policy is **stochastic** and hence **exploratory**. There also exist **deterministic policy gradient (DPG) methods**.

Gradient-ascent algorithm (REINFORCE)

Now, we are ready to present the first **policy gradient algorithm** to find optimal policies. The gradient-ascent algorithm maximizing $J(\theta)$ is

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \nabla_\theta J(\theta) \\ &= \theta_t + \alpha \mathbb{E}[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A)]\end{aligned}$$

The true gradient can be replaced by a stochastic one:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_\pi(s_t,a_t)$$

Furthermore, since q_π is unknown, it can be approximated:

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t,a_t)$$

There are different methods to approximate $q_\pi(s_t,a_t)$, e.g. **Monte-Carlo based** method, **REINFORCE** and **TD**.

Remark 1: How to do sampling?

$$\mathbb{E}_{S\sim d, A\sim\pi}[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A)] \longrightarrow \nabla_\theta \ln \pi(a|s,\theta_t) q_\pi(s,a)$$

How to sample S ?

$S \sim d$, where the distribution d is a long-run behavior under π.

How to sample A?

$A \sim \pi(A|S,\theta)$. Hence, a_t should be sampled following $\pi(\theta_t)$ at s_t.

Therefore, the policy gradient method is **on-policy**.

Remark 2: How to interpret this algorithm?

Since

$$\nabla_\theta \ln \pi(a_t|s_t,\theta_t) = \frac{\nabla_\theta \pi(a_t|s_t,\theta_t)}{\pi(a_t|s_t,\theta_t)},$$

the algorithm can be rewritten as:

$$\theta_{t+1} = \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t,a_t)}{\pi(a_t|s_t,\theta_t)}\right)}_{\beta_t} \nabla_\theta \pi(a_t|s_t,\theta_t)$$

Therefore, we have the important expression:

$$\theta_{t+1} = \theta_t + \alpha\beta_t \nabla_\theta \pi(a_t|s_t, \theta_t)$$

It is a gradient-ascent algorithm for maximizing $\pi(a_t|s_t, \theta_t)$.

> **Intuition: When $\alpha\beta_t$ is sufficiently small**
>
> If $\beta_t > 0$, the probability of choosing(s_t, a_t) is enhanced:
>
> $$\pi(a_t|s_t, \theta_{t+1}) > \pi(a_t|s_t, \theta_t).$$
>
> The greater β_t is, the stronger the enhancement is.
>
> If $\beta_t < 0$, then $\pi(a_t|s_t, \theta_{t+1}) < \pi(a_t|s_t, \theta_t)$.

> The coefficient β_t can well balance **exploration and exploitation**:
>
> First, β_t is **proportional** to $q_t(s_t, a_t)$. If q_t is great, β_t is great. The algorithm intends to enhance actions with greater values.
>
> Second, β_t is **inversely proportional** to $\pi(a_t|s_t, \theta_t)$. If π is small, β_t is large. The algorithm intends to explore actions that have low probabilities.

Recall that

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta_t) q_\pi(s_t, a_t)$$

is replaced by

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t, \theta_t) q_t(s_t, a_t)$$

where $q_t(s_t, a_t)$ is an approximation of $q_\pi(s_t, a_t)$.

- If $q_\pi(s_t, a_t)$ is approximated by Monte Carlo estimation, the algorithm has a specifics name, **REINFORCE**.

Rseudocode

Pseudocode: Policy Gradient by Monte Carlo (REINFORCE)

Initialization: A parameterized function $\pi(a|s,\theta)$, $\gamma \in (0,1)$, and $\alpha > 0$.

Aim: Search for an optimal policy maximizing $J(\theta)$.

For the kth iteration, do

Select s_0 and generate an episode following $\pi(\theta_k)$. Suppose the episode is $\{s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T\}$.

For $t = 0, 1, \ldots, T-1$, do

Value update: $q_t(s_t, a_t) = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$

Policy update: $\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t|s_t,\theta_t) q_t(s_t, a_t)$

$\theta_k = \theta_T$